

JDBC Transaction Management –

1. What is a Transaction?

A transaction is a group of SQL operations executed as a single unit of work.

If all operations succeed → commit the transaction

If any operation fails → rollback the transaction

Example (Real Life)

Bank transfer:

1. Debit money from Account A
2. Credit money to Account B

If step 1 succeeds but step 2 fails → money will be lost.

So both operations must happen **together or not at all**.

2. ACID Properties of Transactions

Transactions follow **ACID** principles.

1. Atomicity

All operations succeed or none succeed.

Example:

A → transfer 500 to B

Either both debit and credit happen or both fail.

2. Consistency

Database must remain in a **valid state** before and after transaction.

Example:

Total bank balance must remain same.

3. Isolation

Transactions should not interfere with each other.

Example:

Two people withdrawing from same account should not cause wrong balance.

4. Durability

Once transaction is **committed**, data is permanently saved.

Even if system crashes.

3. Default Behavior of JDBC

By default **auto-commit mode is ON**.

Meaning:

Every SQL statement is treated as **separate transaction**.

Example

```
INSERT INTO student VALUES(1,'Ram');  
INSERT INTO student VALUES(2,'Shyam');
```

Each statement commits automatically.

4. Why Transaction Management is Needed

We disable auto commit when multiple queries must run together.

Example

Bank transfer:

update account set balance = balance - 500 where id = 1;

update account set balance = balance + 500 where id = 2;

If second query fails → first must be undone.

5. Controlling Transactions in JDBC

JDBC provides methods:

Method	Purpose
setAutoCommit(false)	Disable auto commit
commit()	Save changes permanently
rollback()	Undo changes

6. Steps to Implement Transaction Management

Step 1

Load driver

Step 2

Create connection

Step 3

Disable auto commit

```
con.setAutoCommit(false);
```

Step 4

Execute queries

Step 5

Commit if successful

```
con.commit();
```

Step 6

Rollback if exception occurs

```
con.rollback();
```

7. JDBC Transaction Example (Bank Transfer)

Table

Account

id

name

balance

Program

```
import java.sql.*;
```

```
public class TransactionExample {
```

```
    public static void main(String[] args) {
```

```
        Connection con = null;
```

```
        try {
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test",
                "root",
                "password");
```

```
            // disable auto commit
            con.setAutoCommit(false);
```

```

PreparedStatement debit =
con.prepareStatement(
"update account set balance = balance - ? where id=?");

PreparedStatement credit =
con.prepareStatement(
"update account set balance = balance + ? where id=?");

// debit from account 1
debit.setInt(1,500);
debit.setInt(2,1);
debit.executeUpdate();

// credit to account 2
credit.setInt(1,500);
credit.setInt(2,2);
credit.executeUpdate();

// commit transaction
con.commit();

System.out.println("Transaction Successful");

}
catch(Exception e){

    try{
        con.rollback();
        System.out.println("Transaction Rolled Back");
    }
    catch(Exception ex){
        ex.printStackTrace();
    }

    e.printStackTrace();
}
}
}

```

8. What Happens in This Program

Step by step flow:

1. Connection established
 2. Auto commit disabled
 3. Debit query executed
 4. Credit query executed
 5. If both succeed → commit
 6. If any fails → rollback
-

9. What is Rollback?

Rollback **undoes all operations of current transaction.**

Example:

Debit: 500 from A

Credit: failed

Rollback → restore original balance.

10. Savepoint in JDBC

Savepoint allows **partial rollback**.

Example:

Transaction has 5 queries.

Rollback only to a specific point.

Syntax

```
Savepoint sp = con.setSavepoint();  
con.rollback(sp);
```

Example

```
con.setAutoCommit(false);

stmt.executeUpdate("insert into student values(1,'Ram')");

Savepoint sp = con.setSavepoint();

stmt.executeUpdate("insert into student values(2,'Shyam')");

con.rollback(sp);

con.commit();
```

Result:

Only first insert remains.

11. Transaction Isolation Levels

Isolation level controls **how transactions interact with each other**.

JDBC provides constants.

Level	Description
TRANSACTION_READ_UNCOMMITTED	Dirty reads allowed
TRANSACTION_READ_COMMITTED	Prevents dirty reads
TRANSACTION_REPEATABLE_READ	Prevents non-repeatable reads
TRANSACTION_SERIALIZABLE	Highest isolation

Setting Isolation Level

```
con.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);
```

12. Problems in Concurrent Transactions

1. Dirty Read

Transaction reads **uncommitted data**.

Example

Transaction A updates balance but not committed.

Transaction B reads it.

If A rolls back → B read wrong data.

2. Non Repeatable Read

Reading same row twice gives different result.

Example:

T1 reads balance = 5000

T2 updates balance = 3000

T1 reads again → 3000

3. Phantom Read

New rows appear during transaction.

Example:

SELECT * FROM orders

Later new row inserted by another transaction.

13. Real World Uses of Transaction Management

Used in:

Banking systems

E-commerce payments

Online ticket booking

Inventory management

Money transfer

Order processing

14. Best Practices

Always follow these rules.

1. Disable auto commit when multiple queries involved
2. Use try-catch for rollback
3. Always commit after successful execution
4. Use PreparedStatement to avoid SQL injection
5. Use proper isolation level